

AI工程化系列

AI项目实战指南

AI技术专栏

2026年

目录

1. AI工程化系列 01

1. AI工程化系列 01

AI工程化系列 01 · 大模型部署

AI工程化系列 01

系列一

大模型部署指南 —— 从GPU到云服务

我的观点：

训练大模型是科研问题，部署大模型是工程问题。科研追求精度突破，工程追求系统定性、性能和成本的平衡。一个能训练出SOTA模型的团队，不一定能把模型部署到生产环境。部署的核心挑战不在算法，而在系统工程。

一个真实的案例

某创业公司花费百万训练了一个行业大模型，性能指标很好。

但上线后：

"单卡A100只能跑7B模型"

"10并发请求就把GPU打满了"

"每次部署都要重新下载100GB模型"

后来他们怎么解决的？

答案：

用GPTQ量化把7B模型压缩到INT4（从13GB降到4GB），用vLLM做continuous batching提升吞吐量，用Docker+K8s实现自动化部署和弹性扩缩容。

这不是技术难题，而是工程实践的问题。

1

深度理解：为什么部署这么难？

大模型部署的核心挑战在于三个维度的平衡：

挑战一：计算效率 vs 模型质量

量化可以减少显存占用，但会损失精度；蒸馏可以缩小模型，但需要大量数据和计算资源。如何在两者之间找到平衡点？

挑战二：延 vs 吞吐量

低延要求快速响应（适合对话场景），高吞吐量要求处理更多请求（适合批量任务）。两者往往不可兼得，需要根据业务场景取舍。

挑战三：成本 vs 可用性

更多GPU意味着更高的可用性，但成本也会线性增长。如何设计弹性架构，在流量低谷时节约成本，在流量高峰时保证服务不中断？

2

环境选择：深度对比

环境类型

硬件要求

成本（月）

适用场景

运维复杂度

数据隐私

本地GPU

A100/H100（≥80GB显存）

硬件成本一次性投入，电费约2000元/月

研发测试、隐私敏感场景

高（需维护硬件、网络）

高（数据不离开本地）

云GPU

按需选择（A10、A100、H100）

A100约4–8万元/月，H100约8–15万元/月

生产环境、弹性需求

中（云厂商负责硬件维护）

中（需签署数据合规协议）

API服务

无

按调用量计费，1000次调用约10–50元

快速原型、轻量应用

低（完全托管）

低（数据上传到第三方）

3

模型优化：量化技术深度解析

量化是部署中最常用的优化手段。不同量化方法的对比：

量化类型

精度损失

显存节省

推理速度

适用场景

FP16

很小

约50%

快

对精度要求高的场景

FP8

较小

约75%

很快

高端GPU（H100及以上）

INT8

中等

约75%

非常快

通用场景，推荐首选

INT4

较大

约87.5%

最快

对精度要求不高的场景

实战：GPTQ量化流程深度解析

1. 安装依赖

```
pip install auto-gptq transformers accelerate sentencepiece
```

2. 量化配置详解

```
from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig
```

```
quantize_config = BaseQuantizeConfig(
```

```
bits=4,
```

```
# 量化位数：4或8
```

```
group_size=128,
```

```
# 分组大小，越小精度越高但速度越慢
```

```
desc_act=True,
```

```
# 描述激活值，提升精度
```

```
sym=True,
```

```
# 对称量化，更快
```

```
true_sequential=True
```

```
# 顺序量化，内存效率更高
```

```
)
```

3. 加载并量化模型（首次运行需要）

```
model = AutoGPTQForCausalLM.from_pretrained(
```

```

"lmsys/vicuna-7b-v1.5",

quantize_config=quantize_config
)

model.quantize(dataset, batch_size=4)

model.save_quantized("vicuna-7b-gptq")

# 4. 加载已量化的模型进行推理

model = AutoGPTQForCausalLM.from_quantized(

"vicuna-7b-gptq",

device="cuda:0",

use_triton=True

# 使用Triton加速，性能提升30-50%

)

```

5. 性能对比：7B模型在A100上的表现

FP16: 13GB显存, ~20 tokens/s

INT4: 4GB显存, ~50 tokens/s

显存节省70%, 速度提升150%

4

服务搭建：vLLM深度剖析

vLLM是当前最流行的大模型推理框架，核心优势在于continuous batching。

Continuous Batching原理

传统动态批处理是等一个批次满了再处理，导致资源浪费。vLLM的continuous batching会持续接收新请求，并在每个token生成步骤中动态调度，让GPU始终保持忙碌状态。这可以提升3-5倍的吞吐量。

vLLM架构深度解析：

启动vLLM服务（带详细配置）

```
python -m vllm.entrypoints.api_server \
```

最大并发序列数

总token限制

启用Chat格式

生产环境禁用请求日志

性能监控（Prometheus指标）

http://localhost:8000/metrics 返回：

- vllm_requests_total: 总请求数

- vllm_avg_latency_seconds: 平均延

- vllm_throughput_tokens_per_second: 吞吐量

5

生产架构：高可用设计深度

生产环境架构设计原则：

1.

无状态设计

：每个请求独立处理，支持水平扩展

2.

健康检查

：定期检测服务状态，自动剔除异常节点

3.

熔断降级

：流量过大时降级服务，保护核心功能

4.

灰度发布

：新功能先在小流量验证，再逐步推广

5.

灾备方案

：多可用区部署，防止单点故障

Kubernetes部署示例（关键配置）：

deployment.yaml 关键配置

apiVersion: apps/v1

kind: Deployment

spec:

replicas: 3

至少3个副本保证高可用

strategy:

rollingUpdate:

maxSurge: 1

maxUnavailable: 0

滚动更新时保证零停机

template:

spec:

containers:

resources:

limits:

nvidia.com/gpu: 1

每个pod占用1块GPU

livenessProbe:

httpGet:

path: /health

port: 8000

initialDelaySeconds: 60

启动后60秒开始检查

periodSeconds: 10

readinessProbe:

httpGet:

path: /ready

port: 8000

initialDelaySeconds: 30

6

性能优化：深度调优技巧

技巧一：KV Cache优化

KV Cache是大模型推理的关键。优化策略：

设置合理的max_seq_len，避免浪费显存

使用Flash Attention，加速Attention计算

在多轮对话中复用KV Cache，减少重复计算

技巧二：并行策略选择

模型并行 vs 数据并行

模型并行：把模型参数分到多个GPU（适合超大模型如GPT-3）；数据并行：同一个模型在多个GPU上处理不同数据（适合提升吞吐量）。大多数场景推荐数据并行。

技巧三：流式输出优化

对于对话场景，使用流式输出可以显著降低感知延。vLLM原生支持流式输出：

流式输出调用示例

```
response = client.chat.completions.create(
```

```
model="vicuna-7b",
```

```
messages=[{"role": "user", "content": "Hello"}],
```

```
stream=True
```

```
# 启用流式输出
```

```
)
```

for chunk in response:

```
print(chunk.choices[0].delta.content or "", end="")
```

7

监控与运维：生产环境必备

核心监控指标：

指标类型

具体指标

告警阈值建议

资源使用

GPU显存使用率、GPU利用率、CPU使用率、内存使用率

显存>90%持续5分钟告警

服务性能

请求延、吞吐量、错误率、超时率

延>2s持续1分钟告警

模型质量

困惑度(Perplexity)、生成长度、重复率

困惑度突然上升告警

8

总结：部署的核心思维

部署大模型的核心思维：

1.

量化优先

：先用INT4/INT8量化，再考其他优化

2.

选择合适的框架

：vLLM适合高吞吐量，Triton适合复场景

3.

架构弹性设计

：支持自动扩缩容，应对流量波动

4.

监控闭环

：建立监控-告警-处置的完整流程

5.

持续优化

：根据实际负载不断调整配置

部署不是一次性工作，而是持续迭代的过程。从简单场景开始，逐步优化，才能建立可靠的服务。

系列一：AI工程化系列

01 · 大模型部署指南：从GPU到云服务 ← 你在这里

02 · 模型优化：量化、蒸馏、剪枝实战

03 · API服务搭建：FastAPI+Docker实战

04 · 监控与日志：生产环境运维

05 · RAG实战：构建知识库问答系统

06 · LLM微调：从数据准备到部署